Contextualized Threat Intelligence Prioritization for CTI Analysts
ACE Cyber Applied Research Project



Author(s): Denzel Jefferson

University: Charleston Southern University

Date: 5/04/2026

Executive Summary

Cyber Threat Intelligence (CTI) analysts face growing challenges due to high volumes of unfiltered security alerts generated in real-world environments. These alerts stem from multiple sources, including authentication systems, network traffic, and system-generated events, which creates a high-noise environment that requires constant manual oversight. This overload of security alerts leads to alert fatigue, reduced efficiency, and delayed identification of critical threats (N-able, 2026; Microsoft, n.d.). As a result, analysts must spend significant time manually filtering out irrelevant data before identifying high-risk events (IBM, 2025).

This research investigates whether integrating contextual operational data from Elasticsearch into an automated prioritization system can improve CTI analyst performance. A controlled experimental approach was used to evaluate and compare manual log analysis with an automated prioritization model simulating Information Sharing Platform (MISP) scoring techniques. The testing was implemented within a virtualized testbed environment using Elasticsearch, Kibana, Filebeat, and Docker-based synthetic log generation to replicate CTI conditions.

The results demonstrated significant improvements in efficiency and reduced workload. Manual analysis required an average of 3144 seconds (52.4 minutes) to identify high-priority threats within a dataset of 1,000+ log events. In contrast, automated prioritization reduced this to under one minute, with some test cases completing in seconds. Additionally, the automated model significantly reduced alert noise by filtering large volumes of irrelevant alerts into high-priority events while demonstrating strong performance within the controlled synthetic dataset.

These research findings highlight the effectiveness of contextualized automation by directly addressing alert fatigue and improving operational efficiency in CTI environments. By enabling analysts to rapidly identify and focus on the most critical threats, this approach supports faster decision-making and more effective threat responses. These results provide a strong foundation for future integration into real-world security environments, particularly within Security Operations Centers (SOCs), where scalability and efficiency are essential (Microsoft, n.d.).

Table of Contents

Executive Summary 2

The Research Problem 4

- Introduction & Background 4
- Problem Statement 5
- Research Questions & Hypothesis 5
- Significance & Scope 5-6

Research Methodology 6

- Research Design & Approach 6
- Testbed & Environment 7
- Data Collection & Analysis 8-9

Findings & Recommendations 9

- Results & Findings 9-10
- Discussion & Analysis 10-11
- Actionable Recommendations 11-12
- Limitations of Current R&D Project 12-13

References 14

Appendices 15-21

Table of Figures

Figure 1: Data Pipeline Architecture 7

Table of Tables

Table 1: Project Scope and Boundaries 6

Table 2: Testbed Components and Roles 7

Table 3. Experimental Results Across Multiple Test Runs 9

Table 4: Actionable Recommendations Based on Experimental Findings 11

The Research Problem

Introduction & Background

Cybersecurity environments generate large volumes of log data and threat intelligence alerts that require continuous monitoring and analysis by Cyber Threat Intelligence (CTI) analysts. These alerts originate from multiple sources, including authentication systems, network traffic, and system-level events, creating a high-noise, complex operational environment.

In modern-day workflows, CTI analysts must manually review and analyze all data to determine the severity and relevance of each alert. However, a large percentage of these alerts are low-priority or routine system events that do not require immediate action. Despite this, all alerts must be thoroughly checked, resulting in a demanding process that contributes to inefficiency and an increased workload.

As alert volume continues to increase, this manual approach can become increasingly unsustainable. Analysts are forced to spend excessive time filtering irrelevant data, which delays the identification process of high-priority threats. This issue, commonly referred to as alert fatigue, significantly reduces operational effectiveness and increases the risk of missing critical security events (N-able, 2026).

To address this challenge, this research explores the implementation of an automated contextualized threat prioritization system. By integrating operational environment data and MISP scoring logic, alerts can be automatically filtered and ranked based on relevance and severity, enabling analysts to focus on the most critical threats first (Campbell, 2025).

Existing research shows that alert prioritization is not a new problem in security monitoring. According to Bassey, Idowu, and Ojo (2024), they proposed an alert-prioritization technique that assigned priority based on severity, alert entries, and system criticality, demonstrating that handling alerts should not rely solely on basic severity values or arrival order. Similarly, modern SOC triage workflows emphasize collecting, prioritizing, analyzing, categorizing, and responding to alerts, enabling analysts to focus on the most significant threats first (Radiant Security, 2024).

There are other advanced approaches, such as Automated Alert Classification and Triage (AACT), historical analyst data, machine learning, and supervised learning, all of which are used to improve alert prioritization and automate analyst decisions (Turcotte et al., 2025). These approaches show that contextual prioritization is an active area in real-world research and commercial development.

However, many existing approaches rely mainly on enterprise-level SOC automation platforms, supervised and machine learning models, or historical analyst data. These solutions can offer strong prioritization methods, but some may require large datasets, real-world SOC workflows, commercial funding, or more advanced models, which can make it harder for analysts to interpret. This research is novel because it evaluates a lightweight, logic-based prioritization model integrated directly into an Elasticsearch/Kibana workflow. Instead of relying on a black-box model or full SOAR deployment, the system used weighted contextual scoring based on detectable log features such as severity keywords, user roles, network indicators, and system behavior. This makes the approach easier to explain and validate, especially for resource-constrained environments or small environments.

Problem Statement

CTI analysts are required to manually review and analyze large volumes of unfiltered threat intelligence data, resulting in high alert fatigue and reduced operational efficiency. This manual process slows the identification of critical threats and increases the risk of missing high-priority security events, thereby limiting the overall effectiveness of cybersecurity operations (IBM, 2025).

This research addresses this problem by developing an automated, context-aware prioritization system that filters and ranks alerts based on severity and relevance. This, in turn, will reduce analyst workload while improving the speed of threat identification.

Research Questions & Hypothesis

Research question: How can operational environment data gathered from Elasticsearch be used to automate and improve threat intelligence prioritization within CTI workflows?

Hypothesis: If operational environment data from Elasticsearch is incorporated into an automated MISP-based threat-prioritization system, then the time required for CTI analysts to identify high-priority threats will significantly decrease while maintaining or improving overall prioritization accuracy compared to a manual review process.

Significance & Scope

This research is significant because it directly addresses one of the most critical challenges modern CTI analysts face: alert fatigue (N-able, 2026). As organizations generate increasing volumes of log data, the ability to efficiently prioritize and respond to threats diminishes. Improving alert prioritization not only enhances analyst efficiency but also reduces the likelihood of missing high-risk threats.

This study is particularly relevant for Security Operations Centers (SOCs) and Cyber Threat Intelligence Analysts (CTI), where rapid decision-making and accurate threat identification are critical. By demonstrating measurable improvements in efficiency and workload reduction, this research provides a practical foundation for enhancing existing real-world CTI workflows.

Table 1: Project Scope and Boundaries

Scope of the Project Includes:	Out of Scope:
Simulated CTI analyst environment	Real-world system deployment
Synthetic log generation and ingestion	Integration into enterprise-level SOC platforms
Automated prioritization using contextual scoring logic	Replacement of human analysts

Note. CTI = Cyber Threat Intelligence. SOC = Security Operations Center. This table defines the operational scope and limitations of the proposed research project.

Research Methodology

Research Design & Approach

This research study employs an applied experimental research design and a quantitative approach to evaluate the effectiveness of automated threat prioritization within CTI workflows. The research compares a baseline manual log analysis process with an automated, context-aware prioritization system to measure improvements in operational efficiency.

The experimental design focuses on three primary performance metrics:

- Alert workload reduction
- Time required to identify high-priority threats
- Prioritization accuracy

This approach was selected because it allows a direct comparison between automated and manual workflows within a controlled environment. By having an isolated testing environment, variables, including dataset consistency and real-world workload conditions, can also be isolated so that the study can ensure that observed performance differences are the cause of the selected prioritization method and not external factors.

Testbed & Environment

The research was conducted within a controlled, virtualized testbed environment designed to simulate a realistic CTI analyst workflow while maintaining isolation and reproducibility. The environment was deployed in Oracle VirtualBox and included multiple virtual machines to replicate key components of a cybersecurity data pipeline.

Table 2: Testbed Components and Roles

Component	Role	Technology
Elasticsearch (Elastic, 2025)	Data storage & analysis	Elastic Stack
Kibana	Data Visualization	Elastic Stack
Filebeat	Log Ingestion & forwarding	Elastic
Docker	Synthetic Log Generation	Containers
VirtualBox	Virtualization environment	Oracle VM
MISP (Simulated) (MISP Project, 2024)	Threat scoring logic	Logic-based model

Note. MISP = Malware Information Sharing Platform. The Elastic Stack includes Elasticsearch and Kibana. This table outlines the technologies used in the experimental testbed and their roles.

Each component plays a critical role within the data pipeline, enabling the generation, ingestion, analysis, and visualization of log data. This structure ensures that experiments are consistent, repeatable, and aligned with real-world CTI operational workflows.

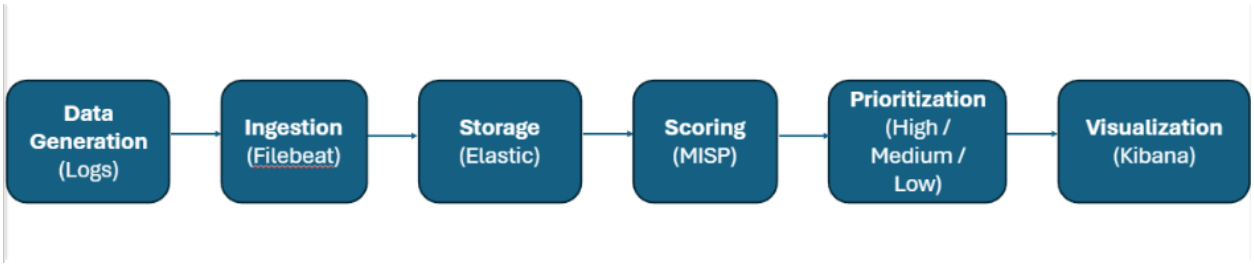


Figure 1: Data Pipeline Architecture. Illustrates the data pipeline used in this study, including log generation, ingestion through Filebeat, storage and analysis within Elasticsearch, application of contextual scoring logic, and final visualization in Kibana.

Data Collection & Analysis

Data for this research were generated using synthetic log-generation techniques designed to simulate realistic CTI analyst environments. Logs were generated using

Linux-based logging commands and Docker-based scripts, resulting in a mix of high-, medium-, and low-priority events. These logs included scenarios such as failed authentication attempts, unauthorized access attempts, and routine system operations to replicate real-world system activity.

The synthetic dataset was designed to replicate a realistic SOC/CTI alert environment by intentionally including a higher proportion of low-priority and routine system alerts. This reflects the real-world problem of analysts reviewing large volumes of alerts, many of which do not pose immediate threats. In the industry, discussions of alert fatigue consistently describe this problem as a result of false positives, missed threats, and analyst workload (Radiant Security, 2024). This synthetic dataset was structured to test whether contextual prioritization could improve the signal-to-noise ratio while reducing the overall number of alerts requiring immediate analyst response.

The generated logs were ingested into Elasticsearch through Filebeat, enabling real-time indexing and analysis (Elastic, 2025). Kibana was the primary interface for visualization, filtering, and log data analysis. This pipeline (Log Generation → Filebeat → Elasticsearch → Kibana) ensured a consistent and reproducible data flow across all test cases.

To enable automated prioritization, a weighted scoring model was implemented using a runtime field within Kibana. This model evaluated each log entry based on multiple contextual factors, including:

- Severity-related keywords (e.g., failed, unauthorized, blocked)
- Network indicators (e.g., internal vs external IP ranges)
- User roles (e.g., admin, root)
- System behavior indicators

Each log was assigned a numerical score that determined its classification as High, Medium, or Low priority. The scoring model was designed to be rule-based and adjustable, unlike a machine-learning model. This design directly supports flexibility and understandability because analysts can manually inspect keywords, network indicators, or user roles and decide how to adjust the final classifications of High, Medium, and Low. Prior alert prioritization research has used severity, entities, identifiers, and asset criticality as factors for ranking alerts, which mirror this study's use of severity indicators, user roles, and network context as prioritization inputs (Bassey et al., 2024). This makes the automated model eligible for a proof-of-concept evaluation while keeping the question of how analysts assign priority rankings open.

The effectiveness of the system was measured by comparing manual log analysis to automated prioritization across multiple test cases. Key performance metrics included

time required to identify high-priority threats, alert volume reduction, and prioritization accuracy. This structured methodology ensured that the results presented were both measurable and aligned directly with the research hypothesis.

Findings & Recommendations

Results & Findings

Table 3. Experimental Results Across Multiple Test Runs

Test Case	Logs	Mean Time (sec)	Median (sec)	Std Dev (sec)	High-Priority Alerts Identified	Key Outcome
TC-001 (Manual Baseline Analysis)	1000	3144	3120	201.69	Manually filtered	High workload, inconsistent
TC-002 (Automated Prioritization)	1065	6.8	6	1.92	10-15 high-priority alerts reviewed	Significant reduction in alert noise, consistent
TC-003 (High Volume Test)	2000	5.8	6	1.8	15-20 high-priority alerts reviewed	Demonstrated scalable performance

Note. Time values are reported in seconds. Results are derived from five experimental runs conducted in a controlled synthetic environment (See Appendix F:). These values represent the number of prioritized alerts the analyst needed to review to identify relevant threats, rather than the total number of high-priority events in the dataset.

These results demonstrate substantial improvements in threat identification performance when using automated vs manual prioritization. Across five experimental runs, manual analysis required an average of 3144 seconds (52.4 minutes) to identify high-priority threats, whereas the automated system achieved an average of 6.8 seconds. This improvement in signal-to-noise ratio represents a 99.8% decrease in identification time, a known challenge in SOC environments (N-able, 2026).

In addition to improved speed, the automated system demonstrated strong consistency across multiple test runs. The standard deviation for manual analysis (201.69 seconds) was significantly higher than that of the automated system (1.92), indicating that manual

performance varied widely and was heavily influenced by human factors, whereas automated prioritization produced stable and predictable results.

Furthermore, the automated system significantly reduced alert volume by filtering approximately 1,000 log events into a high-priority alert category, thereby reducing the number of alerts that required manual analyst review. This improvement in signal-to-noise ratio allows analysts to focus effectively on relevant threats.

While these results visually demonstrate improvements in efficiency and consistency, further analysis is required to fully assess prioritization accuracy and potential system trade-offs, including the potential for future false positives and false negatives. These findings are consistent with setting research indicating that automated prioritization systems significantly reduce analyst workload and improve response efficiency in high-volume alert environments (Bassey et al., 2024; Miller, 2025).

Discussion & Analysis

The results of this study demonstrated that automated contextual prioritization provides a significant improvement over traditional manual analysis methods. The most notable improvement observed was in threat identification time. The time required to identify a high-priority alert in manual analysis was reduced from minutes to mere seconds in automated scenarios. This reduction highlights the effectiveness of automated filtering and ranking in eliminating unnecessary analyst workload (Miller, 2025).

A key factor contributing to this improvement was the integration of contextual scoring by incorporating environmental indicators such as IP ranges, user roles, and severity-based keywords. The system can effectively distinguish between routine system activity and potentially malicious behavior. This enables high-risk alerts to be spotted immediately, rather than being buried within large volumes of irrelevant, unfiltered data.

These research findings validate the research hypothesis that contextualized automation significantly reduces analysis time while improving workflow efficiency. However, evaluating a prioritization system solely on speed is insufficient; classification accuracy and risk management are equally important.

To assess this, the automated system's output was compared against the known classifications of the synthetic dataset. The log generation process explicitly defined high, medium, and low-priority events and provided a baseline for evaluating classification performance. In this research study, all generated high-priority events were correctly identified by the system and listed as True Positives; similarly, there were

no medium- or low-priority events incorrectly listed as high priority (False Positives), and no high-priority events were missed (False Negatives).

While these results indicate perfect classification performance on the controlled dataset, this outcome is influenced by the log structure and the scoring logic. In real-world environments, log data is less clear, and classification errors are more likely. This introduces an important trade-off between efficiency and risk. Specifically, False Negatives pose the greatest concern, as missed high-priority threats can delay CTI detection and response (Miller, 2025). For instance, if a critical alert was assigned a score of 65 below the threshold of 70, it could be categorized as Medium priority and be potentially overlooked.

For this reason, the automated prioritization system should be viewed as an analyst support tool rather than a fully operational scoring system. Human analysts should remain involved in validating high-priority alerts and periodically reviewing low-priority classifications to ensure no critical threats are missed. This hybrid approach will balance the efficiency gains of automation with the need for oversight and prioritized accuracy in real-world CTI workflows.

Actionable Recommendations

Table 4: Actionable Recommendations Based on Experimental Findings

Finding	Recommendation
Manual analysis requires reviewing large volumes of alerts	Implement automated prioritization systems to reduce analyst workload
High alert volume contributes to alert fatigue	Apply contextual filtering to reduce irrelevant alerts
Critical alerts are often buried in large datasets	Use scoring-based ranking models to surface high-priority threats
Increasing log volume impacts efficiency	Deploy scalable ingestion analysis pipelines

Note. Recommendations are derived from observed performance differences between manual and automated prioritization methods within the experimental test cases.

These recommendations are directly derived from the experimental findings and provide a clear framework for improving CTI workflows. Organizations should prioritize

implementing automated prioritization systems that use contextual data, as this approach significantly reduces analyst time and improves operational efficiency. Additionally, improving the signal-to-noise ratio through contextual filtering enables analysts to focus on applied intelligence rather than routine system activity. This not only enhances productivity but also reduces the likelihood of missing critical threats in high-volume environments.

Limitations of Current R&D Project

While this research study's results demonstrate significant improvements in efficiency and reduced workload, several limitations should be considered when interpreting the findings.

First, the dataset used in this research was fully synthetic, generated to simulate realistic cybersecurity events. Although efforts were made to replicate common threat scenarios, synthetic data cannot fully capture the unpredictability, variety, and complexity of real-world threat environments. As a result, system performance may differ greatly when applied to live operational data.

Second, the diversity of generated logs was limited to predefined scenarios such as failed authentication attempts, unauthorized access attempts, and routine system activity. Real-world environments include a broader range of attack patterns and behaviors, which may impact the effectiveness of the automated prioritization model when exposed to more complex threat patterns.

Additionally, the MISP integration within this study was simulated using a rule-based scoring approach rather than a fully implemented MISP platform. While this allowed a controlled test of prioritization logic, it does not fully reflect the dynamic evolution of real-world threat intelligence feeds.

Finally, the testbed environment was constrained by available hardware resources and operated within a controlled virtualized setting. This limits the ability to fully evaluate system performance under real-world conditions such as high data ingestion, distributed environments, and real-time operational constraints.

These limitations highlight the need for further validation and testing in more realistic and complex environments before a full-scale implementation. While the system maintained a consistent performance when processing approximately 1,000 to 2,000 log events, this does not fully replicate real-world environments where Security Operations Centers (SOC) may process millions or billions of events per day.

The prioritization model in this study was implemented using a Kibana runtime field that evaluates the scoring logic at query time rather than during data ingestion. While this

approach provides flexibility and adjustability, it introduces performance limitations as the dataset size increases. As the number of threat events increases, the computational cost of executing scoring logic may grow.

From a computational perspective, this approach does not scale well to large environments and may lead to increased query latency or reduced system responsiveness under heavy log load. In conclusion, the current implementation serves

as a proof of concept, but further optimization and adjustments would be required to ensure scalability in high-volume, real-world CTI environments.

Due to time constraints within the project timeline, the number of experimental iterations was limited to five per test case (See Appendix F) rather than a larger sample size, such as 10-30 iterations. While this sample size is sufficient to demonstrate consistency and performance fluctuations, future work should expand the number of trials to strengthen statistical validity and enable more advanced testing.

Future Work and Path Forward

Future work should focus on expanding the applicability and robustness of the automated prioritization system by directly addressing the limitations identified in this study. A primary focus is on integrating real-world datasets to evaluate system performance against diverse, evolving threat behaviors. This would provide a more accurate assessment of how the automated prioritization model performs in operational environments.

In addition, future work should fully explore integration with the MISP platform to grasp real-time threat intelligence feeds and dynamic scoring systems. This would enhance the system's ability to detect emerging threats and improve prioritization accuracy over time. Scaling the system to support larger and more complex environments is another critical area for future development. This includes testing the system under high data volumes, in real-time processing conditions, and in distributed systems to ensure consistent performance in enterprise-level environments.

Furthermore, incorporating machine learning techniques will enable the automated prioritization model to become more adaptable and intelligent. Unlike rule-based systems, machine learning approaches can identify patterns and relationships in data that may not be explicitly defined, thereby improving overall detection accuracy and reducing false positives. Finally, future efforts should focus on transitioning from a proof-of-concept to an operational system. This includes developing a production-ready product, integrating with existing SOC tools, and conducting real-world validation experiments to support future deployment in cybersecurity operations environments.

References

Assaf, I. (2024, April 17). What is alert triage? soc alert triage process explained. Radiant Security. https://radiantsecurity.ai/learn/soc-alert-triage/

Baker, K. (2025, March 4). What is Cyber Threat Intelligence? [beginner's guide]. CrowdStrike. https://www.crowdstrike.com/en-us/cybersecurity-101/threat-intelligence/

Bassey, C., Idowu, S., & Ojo, C. (2024, July 24). (PDF) alert prioritization techniques in security monitoring: A focus on severity averaging and alert entities. Alert Prioritization Techniques in Security Monitoring. ResearchGate. https://www.researchgate.net/publication/382637241_Alert_Prioritization_Techniques_in_Security_Monitoring_A_Focus_on_Severity_Averaging_and_Alert_Entities

Campbell, C. (2025, April 29). 5 Ways Cyber Threat Intelligence Boosts Cyber Risk Prioritization. Bitsight. https://www.bitsight.com/blog/5-ways-to-boost-cyber-risk-prioritization-with-cti

Elastic. (2025, April). Elastic docs. Elasticsearch Platform - Find real-time answers at scale. https://www.elastic.co/docs

Kosinski, M. (2025, November 17). What is threat intelligence?. IBM. https://www.ibm.com/think/topics/threat-intelligence

Miller, J. (2025, November 4). Automated threat neutralization: How it works and why it matters. BitLyft Cybersecurity. https://www.bitlyft.com/resources/automated-threat-neutralization-how-it-works-and-why-it-matters

MISP Project (2024, October 2). MISP and decaying of indicators primer for indicator scoring in MISP. MISP Threat Sharing. https://www.misp-project.org/misp-training/a.5-bis-decaying-indicators-light-version.pdf

Microsoft. (n.d.). What is Siem? https://www.microsoft.com/en-us/security/business/security-101/what-is-siem

N-able. (2026, February 14). Alert fatigue in cybersecurity: Causes, impact & solutions. https://www.n-able.com/blog/alert-fatigue-cybersecurity

Turcotte, M., & Labrèche, F. (2025, May 14). arxiv. Automated Alert Classification and Triage (AACT): An intelligent system for the prioritization of cybersecurity alerts. https://arxiv.org/html/2505.09843v1

Appendices

Appendix A: Log Generation Script

```
#!/bin/bash

echo "Generating HIGH priority logs..."
for i in {1..100}; do
  logger "Multiple failed login attempts detected from 10.0.0.$i"
  logger "Unauthorized blocked access attempt from 10.0.0.$i"
  logger "Blocked admin login attempt from 10.0.0.$i port 22"
done

echo "Generating MEDIUM priority logs..."
for i in {1..100}; do
  logger "Failed password detected from 192.168.1.$i"
  logger "Invalid user detected from 172.16.0.$i"
  logger "Failed login attempt from 192.168.1.$i"
done

echo "Generating LOW priority logs..."
for i in {1..100}; do
  logger "System operation completed successfully"
  logger "Accepted password from user on network 192.168.10.$i"
  logger "Routine background process completed successfully"
  logger "Accepted user login from 192.168.10.$i"
done

echo "Log Generation Complete."
```

Appendix B: Runtime Field Script

```
String msg = params._source['message'];
int score = 0;

if (msg != null) {
    msg = msg.toLowerCase();

    // Severity Indicators
    if (msg.contains("blocked")) score += 60;
    if (msg.contains("multiple")) score += 50;
    if (msg.contains("denied")) score += 40;
    if (msg.contains("unauthorized")) score += 35;
    if (msg.contains("failed")) score += 30;
    if (msg.contains("invalid")) score += 20;

    // Account Verification
    if (msg.contains("admin")) score += 15;
    if (msg.contains("root")) score += 15;
    if (msg.contains("port 22")) score += 10;

    // Networks
    if (msg.contains("10.0.0")) score += 15;
    if (msg.contains("192.168")) score += 10;
    if (msg.contains("172.16")) score += 10;
    if (msg.contains("127.0.0")) score += 5;

    // Low Risk
    if (msg.contains("accepted")) score -= 10;
    if (msg.contains("success")) score -= 10;
    if (msg.contains("completed")) score -= 10;

    // Final Calculated Priority
    if (score >= 70) {
        emit("HIGH");
    } else if (score >= 30) {
        emit("MEDIUM");
    } else {
        emit("LOW");
    } }
```

Appendix C: Data Generation Report

The synthetic log data used in this study was generated to simulate realistic Cyber Threat Intelligence (CTI) environments within a controlled, virtualized testbed. The data generation process was designed to replicate real-world system behaviors and threat scenarios.

Log data was generated using Linux-based logging commands and automated scripts, resulting in a mix of high-, medium-, and low-priority events. These logs were ingested into Elasticsearch via Filebeat for indexing and analysis, then visualized in Kibana.

The distribution of synthetic log events was designed to replicate a realistic security operations environment where CTI analysts are exposed to a high volume of alerts. Industry research indicates that Security Operations Centers (SOCs) process millions, if not billions, of alerts, with a significant portion classified as low-priority or false positives, contributing to alert fatigue and analyst overload (Radiant Security, 2024; N-able, 2026).

Based on these observations, the dataset was constructed to include a larger proportion of lower-priority events compared to high-priority threats. The distribution used in Test Case 002 is 28% high, 28% medium, and 43% low, demonstrating its balance compared to real-world environments. It provides a controlled, measurable approximation that enables clear evaluation of prioritization performance. This balance ensures the system is tested against relevant threats and background noise while maintaining experimental simplicity.

TC-001 Manual Baseline Analysis

- Total Logs Generated: ~1000
- Distribution:
    - Failed Log Events: 500
    - Multiple Failed Log Events: 300
    - Normal Log Events: 200

- Observation:
    - Logs were unfiltered and required manual review
    - High alert noise and increased analyst workload
- Average Time to Identify Threats: ~3144 seconds (52.4 minutes)

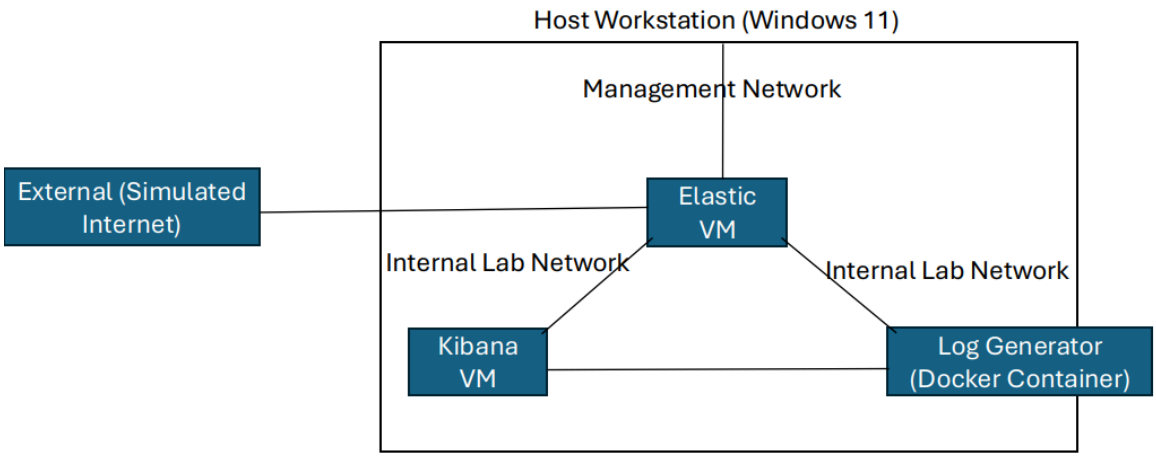
TC-002 Automated Prioritization

- Total Logs Generated: ~1065
- Distribution:
    - High Priority: ~28.2%
    - Medium Priority: ~28.2%
    - Low Priority: ~43.7%
- Observation:
    - Logs were automatically categorized using scoring logic
    - Significant reduction in alert noise
    - High-priority threats were immediately visible
- Average Time to Identify Threats: ~6.8 seconds

TC-003 Hgh Volume Test

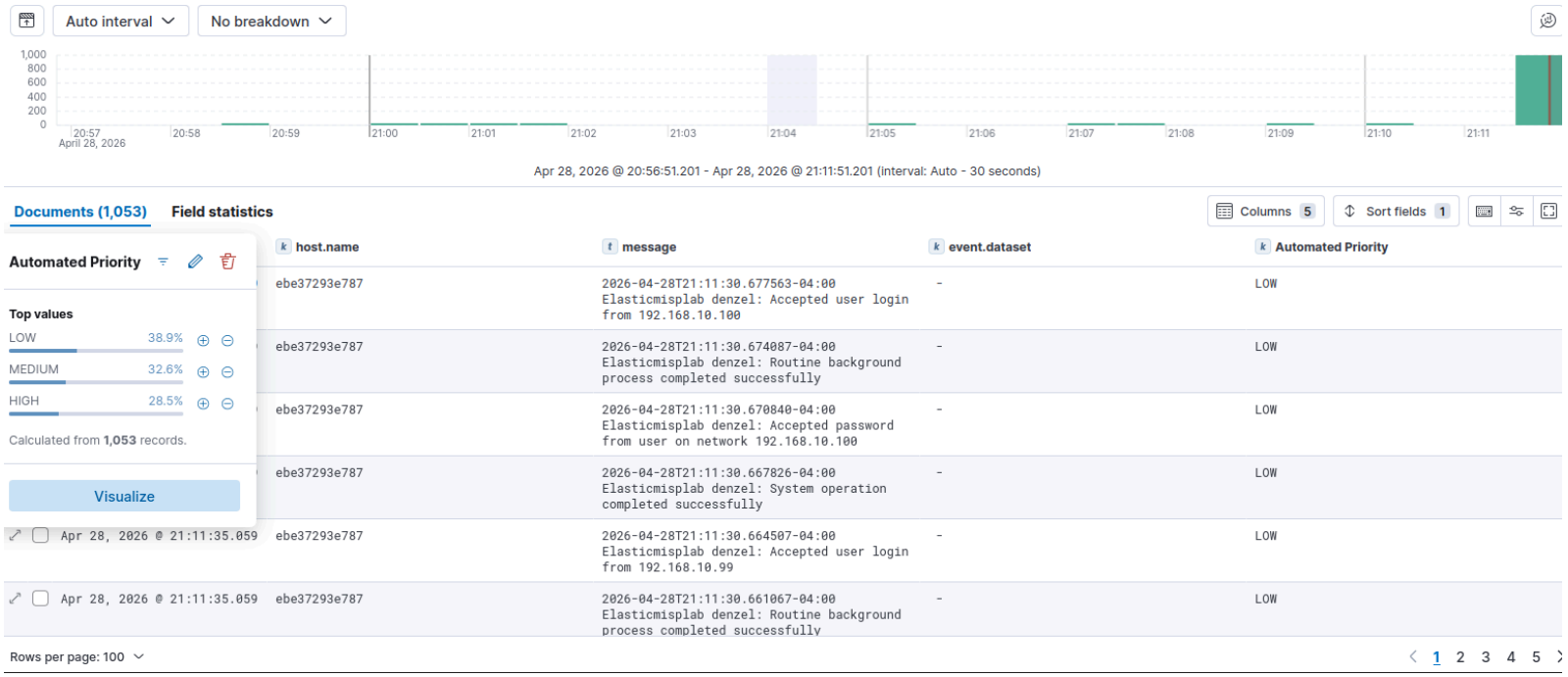
- Total Logs Generated: ~2000
- Distribution:
    - High Priority: 400
    - Medium Priority: 600
    - Low Priority: 1000
- Observation:
    - System maintained performance under increased load
    - High-Priority threats remained easily identifiable
- Average Time to Identify Threats: ~5.8 seconds

Appendix D: Testbed Architecture

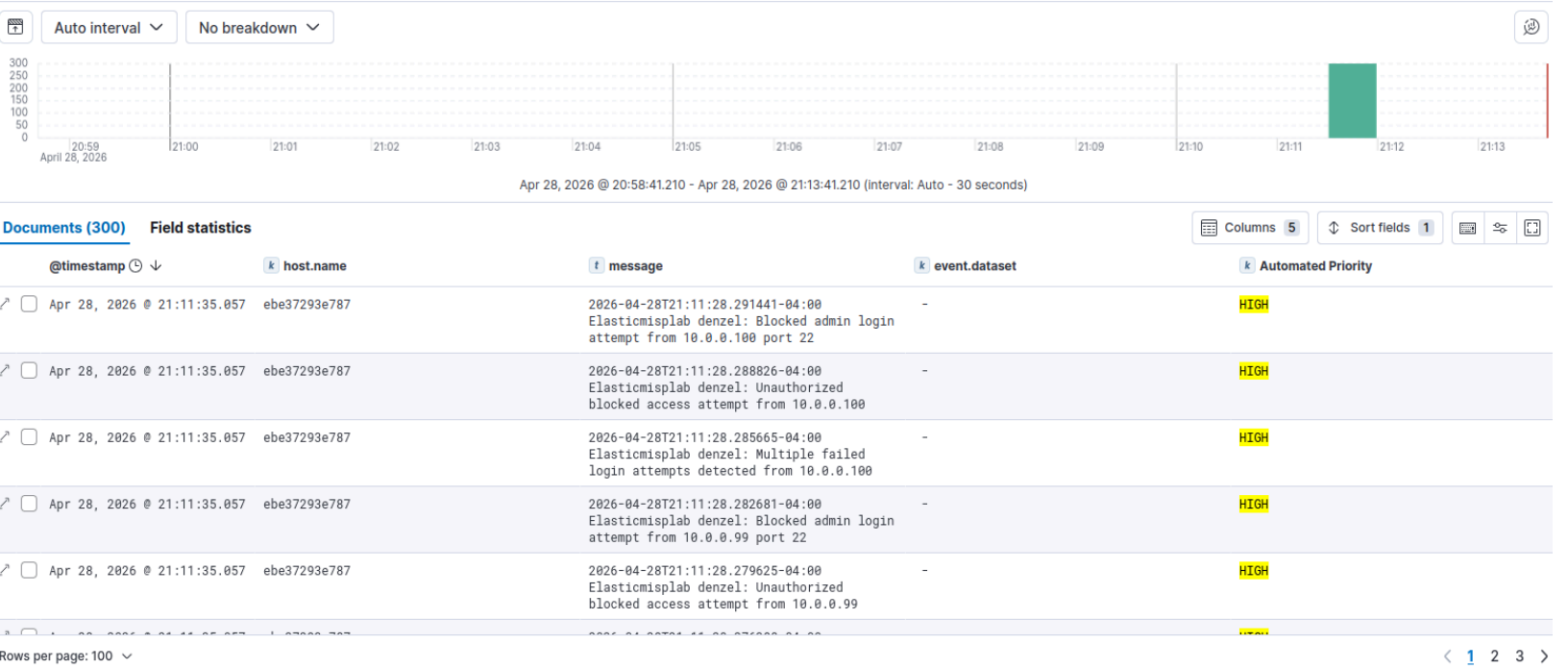


Appendix E: Kibana Dashboards / Results

Before:



After:



Appendix F: Individual Experimental Run Times (Seconds)

Experimental Run	Manual (sec)	Automated (sec)	High-Volume (sec)
1	3120 (52 mins)	10	7
2	3060 (51 mins)	6	4
3	2880 (48 mins)	5	6
4	3420 (57 mins)	6	8
5	3240 (54 mins)	7	4